NPTEL ONLINE CERTIFICATION COURSES



Course Name: Software Project Management

Faculty Name: Dr. Durga Prasad Mohapatra

Department : Computer Sc. & Engg. NIT Rourkela

TOPIC: Project Estimation Techniques cont …



INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

CONCEPTS COVERED

- Halstead's Software Science – An analytical estimation technique.



NPTEL

Categories of Project Estimation Techniques

There are 3 main categories:

- Empirical Estimation: Based on making an educated guess of the project parameters.
  - Example: Expert Judgement technique, Delfi Technique
- Heuristic Estimation: Assume that the relationships that exist among the different project parameters can be satisfactorily modelled using suitable mathematical expressions.
  - Example: COCOMO





Categories of Project Estimation Techniques cont …

- Analytical Estimation: Derive the required results starting with some basic assumptions regarding the project. Unlike Empirical and Heuristic techniques, Analytical techniques do have certain scientific basis.
  - Example: Halstead's Software Science
- Halstead's Software Science is especially useful for estimating software maintenance efforts.
- It outperforms both empirical and heuristics techniques as far as software maintenance efforts is concerned.





Halstead's Software Science: An Analytical Estimation Technique

- It is an analytical technique to estimate:
  - size
  - development effort
  - development time





Halstead's Software Science cont …

- Halstead used a few primitive program parameters
  - number of operators and operands
- Derived expressions for:
  - over all program length,
  - potential minimum volume
  - actual volume,
  - language level,
  - effort, and
  - development time.





Halstead's Software Science cont …

For a given program, let

- η1 be the number of unique operators used in the program
- η2 be the number of unique operands used in the program
- N1 be the total number of operators used in the program
- N2 be the total number of operands used in the program





Guidelines for identifying operators

- Assignment, arithmetic, and logical operators are usually counted as operators
- A pair of parenthesis as well as a block begin-block end pair, are considered as single operators
- A label is considered to be an operator,
  - if it is used as the target of a GOTO statement.
- The constructs **if…then…else…endif** and a **while…do** are considered as single operator





Guidelines for identifying operators cont …

- A sequence (statement termination) operator ‘**;**’ is considered as a single operator.
- Function name in a function call statement is considered as an operator





Guidelines for identifying operands cont…

- Subroutine declarations and variable declarations comprise the operands.
- The function name in a function definition is not counted as operator
- The arguments of the function call are considered as operands
- However, the parameter list of a function in the function declaration statement is not considered as operands.





Operators and Operands for the ANSI C language

- Operators
  - ([. , -> * + - _ ~ ! ++ -- * / % + - << >> < > <= >= != == & ^ | && || = *= /= %= += -= <<= >>= &= ^= |= : ? { ; CASE DEFAULT IF ELSE SWITCH WHILE DO FOR GOTO CONTINUE BREAK RETURN and a function name in a function call
- Operands are those variables and constants which are being used with operators in expressions.
- Note that variable names appearing in declarations are not considered as operands





Examples

Example 1: Consider the expression a = &b;

- Here, a and b are the operands and =, & and ; are the operators

Example 2: Consider the function call statement: **func (a,b);**

- In this func, ‘,’ and ; are considered as operators and variables a, b are treated as operands.





Length and Vocabulary

- The length of a program as defined by Halstead, quantifies total usage of all operators and operands in the program.
- Thus, length N=N1+N2
- The program vocabulary is the number of unique operators and operands used in the program.
- Thus, program vocabulary $\eta = \eta 1 + \eta 2$





Program Volume

- The length of a program depends on the choice of the operators and operands used.
- In other words, for the same programming problem, the length would depend on the programming style.
- This type of dependency would produce different measures of length for essentially the same problem when different programming languages are used.





Program Volume contd…

- Thus, while expressing program size, the programming language used must be taken into consideration.
- Program volume is specified by, $V=N(\log_2 \eta)$
- Idea: Intuitively, the program volume V is the minimum number of bits needed to encode the program.
- To represent η different identifiers uniquely, we need at least $\log_2 \eta$ bits (where η is the program vocabulary).
- Therefore, the volume V represents the size of the program by approximately compensating for the effect of the programming language used.





Potential Minimum Volume

- The potential minimum volume V* is defined as the volume of the most succinct program in which a problem can be coded.
- The minimum volume is obtained when the program can be expressed using a single source code instruction, say a function call like foo();





Potential Minimum Volume

- Thus, if an algorithm operates on input and output data d_1, d_2, …, d_n, the most succinct program would be $f(d_1, d_2, \ldots, d_n)$; for which η1=2, η2=n.
- Therefore, $V^* = (2+ \eta 2)\log_2(2+ \eta 2)$.
- The program level L measures the level of abstraction provided by the programming language.
- $L = V^* / V$.
- Thus, higher the level of a language, the less effort it takes to develop a program using that language.
- This result agrees with the fact that it takes more effort to develop a program in assembly language than to develop a program in a high-level language to solve a problem.





Effort and Time

- The effort required to develop a program can be obtained by dividing the program volume with the level of the programming language used to develop the code.
- Thus, $E = V / L$.
- E is the number of mental discriminations required to implement the program and also the effort required to read and understand the program.
- Thus, programming effort $E = V^2 / V^*$ (Since, $L=V^* /V$)





Effort and Time

- The programming effort varies as the square of the volume.
- Experience shows that E is well correlated to the effort needed for maintenance of an existing program.
- The programmer's time T=E/S,

  where S is the speed of mental discriminations.
- The value of S has been empirically developed from psychological reasoning, and its recommended value for programming application is 18.





Summary

- Halstead's theory tries to provide a formal definition and quantification of such qualitative attributes as
  - program complexity,
  - ease of understanding, and
  - the level of abstraction,

  based on some low-level parameters such as
    - the number of operands, and
    - operators appearing in the program.
- Halstead's software science provides gross estimates of properties of a large collection of software, but extends to individual cases rather inaccurately.





References :

1. B. Hughes, M. Cotterell, R. Mall, *Software Project Management*, Sixth Edition, McGraw Hill Education (India) Pvt. Ltd., 2018.
2. R. Mall, *Fundamentals of Software Engineering*, Fifth Edition, PHI Learning Pvt. Ltd., 2018.






Thank you
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
1951
योग कर्मसु कौशलम्
NPTEL